Mission 3: Multi-Head Attention, RoPE Vector Rotation & Grouped-Query KV-Cache Optimization

Technical Documentation and Evaluation Report

Conversational AI • Winter Semester 2026/27 • Week 3 — Modern Transformer Architectures and Architectural Divergence

Starship Operational Unit: USSE Noosphere Bridge Crew • Waypoint: Pleiades Open Cluster (SEC-03-M45)

Squadron Chomsky

Name	Role
Nikhil Sakharam Thakare	Systems Architect (Lead Engineer) — *adjust roles to your actual split*
Mohammad Salman Sunakal	Communications & Protocol Officer
Vishal Sudarshan PK	Security & Ethics Sentinel
Abdul Haleem Haseeb	Data & Payload Specialist

Project repository: *[insert your Mission 3 repository / squadron branch URL]*

Abstract

This report documents the design and evaluation plan for Mission 3, which restores AURA Subsystem 03 (Distributed Multi-Head Attention Mesh & KV-Cache Transceiver). The work reconstructs the core Transformer self-attention pipeline in pure PyTorch: scaled dot-product multi-head self-attention with causal masking, Rotary Positional Embeddings (RoPE), Grouped-Query Attention (GQA) with a stateful KV-cache, and a hardware-oriented analysis of FlashAttention IO complexity and long-context denial-of-service risk.

The report separates what is derived analytically (for example KV-cache memory footprints, which follow exactly from tensor shapes) from what must be measured on the team's own hardware (for example decoding latency). It does not present unmeasured

numbers as empirical results. The purpose is a reproducible engineering artifact and an honest account of its scope and limitations.

1. Introduction and Problem Statement

The Mission 3 brief describes AURA attempting to process long-context sensor streams with naive, un-tiled Multi-Head Attention. Two costs dominate: the $O(N^2)$ compute and memory of the attention score matrix, and the memory bandwidth required to read the Key-Value cache at every decoding step. Together these saturate the (fictional) photonic memory bus and risk a kernel panic.

The engineering problem is therefore fivefold: (1) implement scaled dot-product attention and validate its variance-preserving scaling; (2) implement RoPE and verify relative-position invariance; (3) implement GQA with head broadcasting; (4) implement a KV-cache and benchmark MHA, GQA and MQA; and (5) analyse the IO behaviour of FlashAttention-style tiling and audit KV-cache exhaustion as a failure mode.

2. Objectives and Assignment Traceability

Assignment requirement	Documented artifact	Status
Multi-head self-attention from scratch (W_Q, W_K, W_V, W_O), head reshaping, causal mask	`01_MultiHead_Attention_Core.py` / `ps03_solution.ipynb`	Design documented; run script to confirm
Variance-preservation unit test	`01_MultiHead_Attention_Core.py`	Test specified (Section 4.4); report observed values after running
`apply_rope(x, freqs_cis)` with θ_i = base^(−2(i−1)/d)	`02_RoPE_Positional_Engine.py`	Design documented; proof in Section 5

Assignment requirement	Documented artifact	Status
Relative-invariance validation (math + empirical)	`02_RoPE_Positional_Engine.py`	Proof provided; empirical check specified
Attention decay plot: RoPE vs ALiBi	`02_RoPE_Positional_Engine.py`	Plot generated by script
`GQAAttention` layer (e.g. H_Q = 32, H_KV = 8)	`03_GQA_and_KVCache_Benchmark.py`	Design documented
`KVCache` class with single-token append	`03_GQA_and_KVCache_Benchmark.py`	Design documented
Benchmark: MHA vs GQA 4:1 vs MQA 32:1 at N ∈ {1024, 4096, 16384, 32768}	`03_GQA_and_KVCache_Benchmark.py`	Memory and bandwidth derived analytically (Section 7); latency to be measured
FlashAttention IO derivation	`04_Memory_Hierarchy_and_Safety_Audit.md`	Derivation in Section 8
KV-cache DoS simulation	`04_Memory_Hierarchy_and_Safety_Audit.md`	Scenario with worked arithmetic in Section 9
Mitigation: PagedAttention and MLA	`04_Memory_Hierarchy_and_Safety_Audit.md`	Evaluation in Section 10

3. System Design

The implementation is split into four independent modules that mirror the bridge-crew roles. The attention core (Part 1) is the foundation; the RoPE engine (Part 2) is a function applied to query and key tensors before the dot product; the GQA layer and KV-cache

(Part 3) extend the core with fewer key/value heads and incremental state; and the audit (Part 4) is analytical and does not depend on running code.

All modules use plain PyTorch tensor operations. The high-level wrapper `nn.MultiheadAttention` is deliberately not used, as the brief requires construction from first principles. Each script is self-contained and runs with `python <filename>.py`, printing formatted terminal output and, where relevant, saving plots.

4. Multi-Head Self-Attention Core (Part 1)

4.1 Scaled dot-product attention

For queries Q, keys K and values V with key dimension d_k:

$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k})\, V$

4.2 Linear projections and head reshaping

Four learned matrices are used: W_Q, W_K, W_V ∈ ℝ^(d_model × d_model) and the output projection W_O. With h heads, head_dim = d_model / h (d_model must be divisible by h). The projected tensors are reshaped as follows:

- Input: `(batch, seq_len, d_model)`
- After projection and view: `(batch, seq_len, num_heads, head_dim)`
- After transpose(1, 2): `(batch, num_heads, seq_len, head_dim)`

Scores have shape `(batch, num_heads, seq_len, seq_len)`. After applying softmax and multiplying by V, the heads are transposed back, made contiguous, concatenated to `(batch, seq_len, d_model)`, and passed through W_O.

4.3 Causal masking

For autoregressive decoding, position i must not attend to positions j > i. A lower-triangular boolean mask (for example `torch.tril(torch.ones(T, T))`) is used to fill scores above the diagonal with −∞ before softmax, so those positions receive exactly zero attention weight. A useful check is that the attention weights above the diagonal are all zero and each row still sums to 1.

4.4 Variance-preservation unit test

Assume the components of q and k are independent with zero mean and unit variance. The dot product $q \cdot k = \Sigma_{i=1}^{d_k} q_i k_i$ is a sum of d_k independent terms. Each term has $E[q_i k_i] = 0$ and $\mathrm{Var}(q_i k_i) = E[q_i^2]E[k_i^2] = 1$, so $\mathrm{Var}(q \cdot k) = d_k$. Dividing by $\sqrt{d_k}$ scales the variance by $1/d_k$, giving $\mathrm{Var}(q \cdot k / \sqrt{d_k}) = 1$.

The empirical test samples many (q, k) pairs from N(0, 1) for several d_k values (for example 16, 64, 256, 1024), computes the sample variance of the raw and scaled dot products, and asserts that the scaled variance is close to 1.0 within a statistical tolerance while the unscaled variance is close to d_k. This matters because large-variance logits push softmax into saturated regions where gradients vanish. Observed values should be copied from the terminal output into the final submission.

5. Rotary Positional Embeddings (Part 2)

5.1 Definition

RoPE (Su et al., 2021) encodes position by rotating pairs of feature dimensions. The head dimension d is split into d/2 two-dimensional planes. Plane i (i = 1, …, d/2) has rotation frequency

$\theta_i = \mathrm{base}^{-2(i-1)/d}$, with base = 10000 by default.

For a token at position m, the pair (x_{2i-1}, x_{2i}) is rotated by angle $m \cdot \theta_i$. Treating the pair as a complex number $z = x_{2i-1} + j \cdot x_{2i}$, this is multiplication by $e^{j \cdot m \cdot \theta_i}$. `apply_rope(x, freqs_cis)` precomputes `freqs_cis = exp(j · m · θ_i)` for all positions and frequencies, views the last dimension of x as complex numbers, multiplies, and views back as real. It is applied to Q and K only, never to V.

5.2 Relative-position invariance (proof)

In a single plane, let R_m be the 2×2 rotation matrix by angle $m \cdot \theta$. Rotation matrices satisfy $R_m^T = R_{-m}$ and $R_a R_b = R_{a+b}$. Then

$\langle R_m q, R_n k \rangle = q^T R_m^T R_n k = q^T R_{n-m} k$.

The result depends on q, k and only the difference $n - m$, never on the absolute positions. The same holds for every plane, so for the full vector the inner product is a function $g(q, k, m - n)$. Consequently, for any shift δ,

$\langle \mathrm{RoPE}(q, m + \delta), \mathrm{RoPE}(k, n + \delta) \rangle = q^T R_{(n+\delta)-(m+\delta)} k = \langle \mathrm{RoPE}(q, m), \mathrm{RoPE}(k, n) \rangle$.

5.3 Empirical validation

The script draws random q and k, evaluates the inner product at several (m, n) pairs and multiple shifts δ, and asserts agreement within floating-point tolerance (for example `torch.allclose` with atol ≈ 1e-5 in float32). It should also confirm that rotation preserves vector norm.

5.4 RoPE decay versus ALiBi

RoPE has no explicit penalty term; its long-range decay is an emergent property of the rotation frequencies, because high-frequency planes decorrelate quickly with distance while low-frequency planes remain aligned. ALiBi instead adds a fixed linear penalty −m_h · |i − j| to the logits, where m_h is a per-head slope. The script plots attention weight (or logit contribution) against relative distance for both schemes, and a heatmap over (query position, key position). The expected qualitative picture is a smooth, oscillating decay for RoPE and a monotonic linear-in-logit decay for ALiBi. The exact curve depends on the random vectors and slopes chosen, so the figure should be described from the generated plot rather than from theory alone.

5.5 Context-length extrapolation

The directive also asks for an evaluation of extrapolation. Positions beyond the training length give rotation angles the model never saw. Frequency-scaling remedies such as position interpolation (dividing positions by a scale factor) or a larger base (NTK-style scaling) are the standard mitigations. These are discussed here conceptually; any claim about downstream model quality would require training or evaluating a real model and is not made.

6. Grouped-Query Attention and KV-Cache (Part 3)

6.1 GQA layer

In GQA, H_Q query heads share H_KV key/value heads, with group size G = H_Q / H_KV. Special cases: H_KV = H_Q is standard MHA, and H_KV = 1 is MQA. The layer projects queries to H_Q heads but keys and values to only H_KV heads, so W_K and W_V shrink by a factor of G. Before computing scores, K and V are broadcast along the head axis, using `torch.repeat_interleave(k, G, dim=1)` or an `expand` + `reshape`, so that query head j uses key/value head ⌊j / G⌋. The layer must assert that H_Q is divisible by H_KV.

6.2 KVCache class

During autoregressive decoding, keys and values of earlier tokens never change, so recomputing them each step is wasted work. The `KVCache` class stores K and V tensors of shape `(batch, H_KV, t, head_dim)` and, on each step, appends the new single-token slice along the sequence axis (either by `torch.cat` or by writing into a preallocated buffer with a position pointer). Preallocation avoids repeated reallocation. Importantly, the cache stores the **H_KV** heads, before broadcasting, which is where GQA saves memory. For RoPE models the cached keys are stored after rotation at their original positions.

A correctness test should check that incremental decoding with the cache produces the same outputs (within tolerance) as a full forward pass over the whole sequence with a causal mask.

6.3 Complexity

Without a cache, generating N tokens recomputes attention over the full prefix at each step. Step t costs $O(t^2)$ for scores, so the total is $\Sigma\, O(t^2) = O(N^3)$. With a cache, the K/V projections for each new token are computed once (O(1) per token with respect to sequence length), while the attention step itself still reads the t cached entries, O(t) per token. The brief's "O(1) token generation" is therefore correct for the projection work and the *recomputation* avoided, but attention over the cache remains linear in context length, which is exactly why cache memory bandwidth becomes the bottleneck studied here.

7. Benchmark: Memory, Bandwidth and Latency

7.1 Method and assumptions

The KV-cache footprint is fully determined by the model configuration:

bytes per token = 2 (K and V) × L × H_KV × d_head × bytes_per_element

The reference configuration assumed here is LLaMA-3-8B-like: L = 32 layers, H_Q = 32, d_head = 128, fp16 (2 bytes), batch size 1. This gives 2 × 32 × H_KV × 128 × 2 bytes = 16,384 × H_KV bytes per token, i.e. 0.5 MB per token for MHA (H_KV = 32), 0.125 MB for GQA 4:1 (H_KV = 8) and 0.015625 MB for MQA 32:1 (H_KV = 1). Here 1 MB = 2^{20} bytes and 1 GB = 2^{30} bytes.

Memory bandwidth traffic assumes that each decoded token reads the entire cache once, at 50 tokens/s: bandwidth = footprint × 50. This is a lower bound on cache traffic only; it excludes weight reads and writes of the new token.

7.2 KV-cache footprint (MB), full model

N	MHA (32:32)	GQA 4:1 (32:8)	MQA 32:1 (32:1)
1,024	512	128	16
4,096	2,048	512	64
16,384	8,192	2,048	256
32,768	16,384	4,096	512

7.3 Cache read traffic at 50 tokens/s (GB/s), full model

N	MHA	GQA 4:1	MQA 32:1
1,024	25.00	6.25	0.78
4,096	100.00	25.00	3.13
16,384	400.00	100.00	12.50
32,768	800.00	200.00	25.00

The reduction factors are exactly 4× for GQA 4:1 and 32× for MQA 32:1, independent of N. Note that MHA at N = 32,768 would demand 800 GB/s for cache reads alone, which exceeds the HBM bandwidth of many accelerators; this is the quantitative form of the "saturated memory bus" in the mission narrative.

7.4 Decoding latency per token (ms)

Latency depends on hardware, dtype, kernel implementation and whether the benchmark runs on CPU or GPU, so it cannot be derived analytically. The script measures it with warm-up iterations, multiple timed repetitions, and `torch.cuda.synchronize()` before reading the timer when a GPU is used.

N	MHA (ms)	GQA 4:1 (ms)	MQA 32:1 (ms)
1,024	*measure*	*measure*	*measure*
4,096	*measure*	*measure*	*measure*
16,384	*measure*	*measure*	*measure*
32,768	*measure*	*measure*	*measure*

Evidence status: the latency cells are intentionally left blank. Paste the values from your own run of `03_GQA_and_KVCache_Benchmark.py` and record the device, PyTorch version and dtype beside the table. If a full 32-layer cache at N = 32,768 does not fit on your device (MHA needs 16 GB for the cache alone), benchmark a single layer and state that scaling explicitly. Expect latency to decrease from MHA to GQA to MQA at long N because decode attention is memory-bound, but report what you actually observe.

8. FlashAttention IO Complexity Analysis (Part 4)

8.1 GPU memory hierarchy

A GPU has large but comparatively slow High-Bandwidth Memory (HBM) and small but very fast on-chip SRAM. Let M be the SRAM size in elements, N the sequence length and d the head dimension. Attention is typically limited by HBM reads and writes rather than arithmetic, so the relevant cost measure is the number of HBM accesses.

8.2 Standard attention

Standard attention materialises the full N × N matrices in HBM:

1. Read Q, K (O(Nd)); compute $S = QK^T$ and write S to HBM: $O(N^2)$.
2. Read S; compute P = softmax(S) and write P: $O(N^2)$.
3. Read P and V; compute O = PV and write O: $O(N^2 + Nd)$.

Total HBM traffic: $\Theta(Nd + N^2)$. Memory for S and P is also $\Theta(N^2)$.

8.3 FlashAttention tiling

FlashAttention splits Q into blocks of B_r rows and K, V into blocks of B_c rows, and uses the online softmax (running row maximum and running normaliser) so that the N × N matrix is never written to HBM. Choosing $B_c = \lceil M / 4d \rceil$ and $B_r = \min(\lceil M / 4d \rceil, d)$

ensures that the Q, K, V, O blocks and the small score tile fit in SRAM of size M.

The outer loop runs over $T_c = \lceil N / B_c \rceil = \Theta(Nd / M)$ column blocks. Each K, V block is loaded from HBM once (total $\Theta(Nd)$). For each column block, the inner loop streams all of Q and the running O, giving $\Theta(Nd)$ HBM traffic per outer iteration. Total:

HBM accesses $= \Theta(Nd \cdot T_c) = \Theta(Nd \cdot Nd / M) = \Theta(N^2d^2 / M)$.

Including the one-time loads, this is $O(Nd + N^2d^2 / M)$, matching the brief. For typical values ($d = 64$–128, $M \approx 100$ KB $\approx 50{,}000$ fp16 elements) the ratio d^2/M is much less than 1, so FlashAttention performs several times fewer HBM accesses than standard attention, and activation memory falls from $\Theta(N^2)$ to $\Theta(N)$.

8.4 A caveat on the brief's O(N) claim

The overview in the brief says traffic falls "from $O(N^2)$ to $O(N)$". Strictly, the HBM *traffic* remains quadratic in N, with a much smaller constant (d^2/M); it is the extra *memory* for the attention matrix that becomes linear. This report uses the precise statement above. FlashAttention also computes exact attention, not an approximation, and FlashAttention-3 additionally exploits Hopper-specific asynchrony and low-precision features, which are hardware optimisations beyond the IO argument.

9. KV-Cache Denial-of-Service Scenario

9.1 Threat model

The adversary is an ordinary client, or many coordinated clients, of the inference service. They open many concurrent multi-turn dialogues and keep each one alive while steadily lengthening the context, either by sending long prompts or by eliciting long outputs. The service imposes no per-session token cap or admission control. This is a resource-exhaustion attack and requires no special access.

9.2 Worked example (illustrative arithmetic)

Assume one 80 GB GPU serving the LLaMA-3-8B-like model of Section 7 in fp16. Weights occupy about 16 GB. Reserving roughly 4 GB for activations and runtime overhead leaves about 60 GB for KV-cache.

Attention type	Cache per 32k-token session	Concurrent sessions that fit in ~60 GB
MHA	16 GB	3
GQA 4:1	4 GB	15
MQA 32:1	0.5 GB	120

An attacker who opens only four full-length sessions on an MHA deployment, or sixteen on GQA, can exhaust the cache. Once the allocator cannot satisfy the next request for cache growth, the framework raises a CUDA out-of-memory error. Depending on the serving stack, this can abort the in-flight batch, evict or crash other users' requests, or stall the service. Legitimate users on the same device are affected, so this is an availability attack, not merely a performance degradation. The numbers are a model calculation; they were not obtained by attacking a real system, and the exact capacity depends on the serving framework and its overheads.

9.3 Fragmentation amplifies the problem

If the server reserves a contiguous block per request sized to the maximum possible context, then short requests waste most of their reservation (internal fragmentation) and variable-sized allocations leave unusable gaps (external fragmentation). The effective number of concurrent sessions is then lower than the ideal table above.

10. Mitigation Strategies

10.1 PagedAttention (vLLM)

PagedAttention stores the KV-cache in fixed-size blocks (pages) allocated on demand and tracked through a per-sequence block table, analogous to virtual memory paging. Blocks need not be contiguous. Benefits: near-elimination of internal and external fragmentation (the vLLM paper reports that earlier systems wasted a large fraction of cache memory and that paging reduces waste to a few percent), copy-on-write sharing of common prefixes (for example a shared system prompt), and easy preemption or swapping of whole sequences. Limitations: it improves memory *utilisation*, not total memory *demand*, so a determined attacker can still exhaust the pool and admission control is still required. The block-table indirection also adds modest kernel complexity.

10.2 Multi-Head Latent Attention (MLA, DeepSeek-V3)

MLA reduces the cache by projecting keys and values into a shared low-rank latent vector and caching only that latent (plus a small decoupled RoPE key component), then reconstructing or absorbing the up-projections at attention time. In DeepSeek-V3 the cached state per token per layer is on the order of 576 values (a 512-dimensional latent plus a 64-dimensional rotary key), compared with tens of thousands of values for standard MHA at comparable scale. This attacks the demand side directly. Limitations: it is an architectural choice fixed at training time, so it cannot be retrofitted onto an existing MHA or GQA checkpoint without retraining, and it adds implementation complexity for RoPE compatibility.

10.3 Complementary operational controls

Architectural defences do not replace policy controls. Recommended controls for the *USSE Noosphere* mainframes are a hard per-session and per-user token limit, admission control that rejects or queues new sessions when projected cache use exceeds a threshold, idle-session eviction with timeouts, quantised (for example 8-bit) KV-cache where quality permits, sliding-window attention for appropriate layers, per-tenant quotas, and monitoring with alerting on cache occupancy.

10.4 Comparison

Defence	Attacks which problem	Reduces total demand?	Needs retraining?
GQA / MQA	Cache size per token	Yes (4× / 32× here)	Yes (or uptraining)
PagedAttention	Fragmentation, sharing	No (improves utilisation)	No
MLA	Cache size per token	Yes (large)	Yes
Quotas and admission control	Abuse and overload	Bounds it	No

11. Test Strategy and Verification

The verification plan consists of regression checks, not a proof of correctness:

- **Attention core:** output shape equals `(batch, seq_len, d_model)`; causal mask zeroes all upper-triangular weights and rows sum to 1; the output at position i is unchanged when later tokens are modified (causality); results match `torch.nn.functional.scaled_dot_product_attention` within tolerance as a reference check only (not used in the implementation).
- **Variance test:** scaled dot-product variance close to 1.0 and unscaled close to d_k for several d_k values.
- **RoPE:** inner-product invariance under shifts δ; norm preservation; shape handling for odd and even batch/head layouts.
- **GQA/KV-cache:** with H_KV = H_Q the GQA layer matches the MHA layer given identical weights; cached incremental decoding matches full-sequence decoding; cache length grows by exactly one per step.
- **Benchmark:** analytical footprint formula agrees with `tensor.element_size() * tensor.nelement()` of the allocated cache.

When the implementation changes, rerun all scripts and regenerate tables and plots so the evidence corresponds to the current code.

12. Hazard Analysis

The main risks are: shape and transposition errors that silently produce plausible but wrong attention; numerical issues such as float16 overflow in unscaled logits or softmax of rows that are fully masked (producing NaN); off-by-one position errors when decoding with a cache and RoPE; benchmark artefacts such as missing GPU synchronisation, no warm-up, or CPU-versus-GPU mix-ups; misinterpreting analytical bandwidth as measured bandwidth; and overclaiming from a small educational prototype. Mitigations are the tests above, explicit assertions on shapes and divisibility, computing softmax in float32 where needed, and labelling analytical and measured figures separately. The code is educational and must not be used for real navigation or other safety-critical operations.

13. Reproducibility and Repository Use

The expected structure is `docs/weeks/week-03/problem_set/` containing `01_MultiHead_Attention_Core.py`, `02_RoPE_Positional_Engine.py`, `03_GQA_and_KVCache_Benchmark.py`, `04_Memory_Hierarchy_and_Safety_Audit.md` and `ps03_solution.ipynb`. Each script runs with `python <filename>.py`. PyTorch 2.4 or later is required; the benchmark plots need matplotlib. A GPU is optional but recommended for realistic latency numbers; no API key is needed. Commit all deliverables to the squadron branch with the message:

```
feat(ps03): restore AURA Subsystem 03 Spatial Attention Mesh (SEC-03-M45)
```

14. Limitations and Future Work

The prototype is a small educational implementation: models are untrained, so no claim is made about language quality or long-context accuracy; the bandwidth figures are a first-order analytical model; latency depends on the measurement hardware; and the DoS scenario is a calculation, not an experiment on production infrastructure. Future work could add a fused or tiled attention kernel to measure the FlashAttention effect directly, a paged KV-cache prototype, an MLA layer, KV-cache quantisation, and evaluation of RoPE scaling methods on a real checkpoint. Such additions should be included only when their behaviour can be tested and documented.

15. Conclusion

The project reconstructs the key components of modern Transformer attention: multi-head self-attention with a validated scaling factor, RoPE with a proven relative-position property, GQA with a stateful KV-cache, and a memory-hierarchy analysis of FlashAttention-style tiling. The analytical results show that KV-cache memory and read traffic scale linearly with the number of KV heads (4× lower for GQA 4:1, 32× lower for MQA 32:1), and that long-context caches are a realistic denial-of-service surface that architectural and policy defences should address together. Measured latency and plots remain to be added from the team's own runs.

References

1. Conversational AI course assignment, *Mission 3: Multi-Head Attention, RoPE Vector Rotation & Grouped-Query KV-Cache Optimization*, Winter Semester 2026/27, supplied briefing.
2. Vaswani, A. et al. (2017). "Attention Is All You Need." *NeurIPS 30*.
3. Su, J. et al. (2021). "RoFormer: Enhanced Transformer with Rotary Position Embedding." arXiv:2104.09864.
4. Press, O., Smith, N. A., Lewis, M. (2022). "Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation." ICLR 2022 (ALiBi).
5. Shazeer, N. (2019). "Fast Transformer Decoding: One Write-Head is All You Need." arXiv:1911.02150 (MQA).
6. Ainslie, J. et al. (2023). "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints." EMNLP 2023.
7. Dao, T. et al. (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." NeurIPS 2022.
8. Shah, J. et al. (2024). "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision." arXiv:2407.08608.
9. Kwon, W. et al. (2023). "Efficient Memory Management for Large Language Model Serving with PagedAttention." SOSP 2023.
10. DeepSeek-AI (2024). "DeepSeek-V3 Technical Report." arXiv:2412.19437 (MLA).
11. Squadron Chomsky, *Mission 2 Technical Report* (format reference).